Servicio Nacional de Aprendizaje – SENA

Centro Agroempresarial y Desarrollo Pecuario del Huila – Garzón

Tecnólogo en Análisis y Desarrollo de Software (228118)

Definir estándares de codificación, de acuerdo con la plataforma de desarrollo elegida

Evidencia GA7-220501096-AA1-EV02

Proyecto: Tienda Deportes · Node.js, Express, React y MySQL

Aprendiz: [Nombre completo del aprendiz]

Ficha: [Número de ficha]

Instructor: [Nombre del instructor]

Garzón, Huila · Noviembre de 2026

Contenido

1. Introducción

2. Objetivos

    2.1 Objetivo general

    2.2 Objetivos específicos

3. Plataforma de desarrollo y guías de referencia

4. Reglas generales

    4.1 Idioma de los identificadores

    4.2 Estilos de escritura

    4.3 Formato

5. Estándares para Node.js y Express

    5.1 Organización de carpetas y módulos

    5.2 Nombramiento de clases

    5.3 Nombramiento de funciones y métodos

    5.4 Nombramiento de variables y constantes

    5.5 Declaración de clases

    5.6 Declaración de funciones y acceso a datos

    5.7 Rutas, controladores y manejo de errores en Express

    5.8 Comentarios

6. Estándares para React

7. Estándares para la base de datos

8. Estándares para la API REST

9. Herramientas para verificar los estándares

10. Lista de verificación antes de cada commit

11. Conclusiones

12. Referencias

1. Introducción

Un estándar de codificación es el conjunto de reglas que todo el equipo sigue al escribir código: cómo se nombran variables, funciones, clases y módulos, cómo se ordenan y comentan, y cómo se da formato a cada archivo. Cuando todo el código se escribe igual, se lee como si lo hubiera hecho una sola persona, los errores se encuentran más rápido y es más fácil revisar los cambios en GitHub.

Este informe define los estándares de la Tienda Deportes para la plataforma seleccionada en la evidencia AA1-EV01: Node.js con Express en el back-end, React en el front-end y MySQL como base de datos. Como todo el proyecto está escrito en JavaScript, un solo conjunto de reglas cubre el servidor y el cliente. Las reglas parten de guías reconocidas (Airbnb JavaScript Style Guide, Node.js Best Practices y la documentación de Express y React) y se adaptan al proyecto.

2. Objetivos

2.1 Objetivo general

Definir los estándares de codificación que se aplicarán en la construcción del software de la Tienda Deportes, de acuerdo con las reglas de las tecnologías de la plataforma de desarrollo elegida.

2.2 Objetivos específicos

- Establecer las reglas de nombramiento de carpetas, módulos, clases, funciones, variables y constantes.
- Definir la estructura y el orden de declaración de las clases y las funciones.
- Fijar las convenciones de formato, comentarios y manejo de errores en Node.js y Express.
- Definir los estándares para los componentes de React, la base de datos y las URL de la API.
- Seleccionar herramientas que verifiquen automáticamente el cumplimiento de los estándares.

3. Plataforma de desarrollo y guías de referencia

Capa	Tecnología	Guía de estilo de referencia
Lenguaje	JavaScript moderno (ES2023) con módulos ES	Airbnb JavaScript Style Guide

Capa	Tecnología	Guía de estilo de referencia
Back-end	Node.js 24 (LTS), Express 5, mysql2, EJS	Node.js Best Practices y documentación de Express
Front-end	React con Vite	Documentación oficial de React
Base de datos	MySQL 8	SQL Style Guide (Simon Holywell)
API	REST sobre HTTP con JSON	Buenas prácticas de diseño de API REST
Versionamiento	Git y GitHub	Conventional Commits (definido en la AA1-EV01)

4. Reglas generales

4.1 Idioma de los identificadores

Los nombres se escriben en **español**, igual que las tablas de la base de datos y los requerimientos, para que el código hable el mismo idioma del negocio: Producto, calcularTotal, listarPedidos. Se exceptúan las palabras del lenguaje y de las librerías (req, res, next, use, get, Router).

- **Sin tildes ni ñ** en los identificadores: anio en lugar de año, descripcion en lugar de descripción. Los textos que ve el usuario sí llevan tildes.
- **Nombres completos y descriptivos:** cantidadDisponible y no cd o cantDisp.
- **Sin números mágicos:** los valores fijos se declaran como constantes con nombre.

4.2 Estilos de escritura

Estilo	Forma	Se usa para	Ejemplo
camelCase	Primera palabra en minúscula, las demás con mayúscula inicial	Variables, funciones, métodos, props	precioUnitario, buscarPorId()
PascalCase	Todas las palabras con mayúscula inicial	Clases y componentes de React	Producto, TarjetaProducto
UPPER_SNAKE_CASE	Mayúsculas separadas por guion bajo	Constantes y variables de entorno	STOCK_BAJO, DB_HOST
kebab-case	Minúsculas separadas por guion	Carpetas, URL de la API, ramas	detalle-pedido, /api/detalle-pedidos

Estilo	Forma	Se usa para	Ejemplo
snake_case	Minúsculas separadas por guion bajo	Tablas y columnas nuevas de la base de datos	detalle_pedido, id_pedido

4.3 Formato

Regla	Valor
Indentación	2 espacios, sin tabuladores
Longitud máxima de línea	100 caracteres
Llaves	Abren en la misma línea y siempre se usan, aunque el bloque tenga una sola línea
Comillas	Dobles para textos; plantillas `${}` para unir textos con variables
Punto y coma	Obligatorio al final de cada sentencia
Comparaciones	Siempre === y !==, nunca == ni !=
Declaraciones	const por defecto, let solo si el valor cambia, nunca var; una por línea
Módulos	Módulos ES: import y export ("type": "module" en package.json); nunca require
Codificación y fin de línea	UTF-8, LF

Nota: Prettier aplica automáticamente la indentación, las comillas, el punto y coma y la longitud de línea al guardar el archivo.

5. Estándares para Node.js y Express

5.1 Organización de carpetas y módulos

El back-end se organiza por **capas**. Cada capa es una carpeta en minúsculas y cada archivo indica el recurso y la capa: **recurso.capa.js**, en singular y minúsculas. Así, al ver el nombre se sabe qué contiene sin abrirlo.

```
backend/
├── app.js                      crea la aplicación de Express
├── servidor.js                 arranca el servidor en el puerto
└── src/
    ├── config/baseDatos.js     pool de conexiones a MySQL
    ├── rutas/producto.rutas.js
    ├── controladores/producto.controlador.js
    ├── servicios/producto.servicio.js
    ├── modelos/Producto.js          clase de la entidad
    ├── modelos/producto.modelo.js   consultas SQL
    ├── middlewares/manejarErrores.js
    ├── errores/StockInsuficienteError.js
    ├── vistas/productos/lista.ejs
    └── utils/formatoMoneda.js
```

Capa	Responsabilidad	Archivo de ejemplo
rutas	Asocian cada URL y método HTTP con su controlador	producto.rutas.js
controladores	Leen la petición (req), llaman al servicio y responden (res)	producto.controlador.js
servicios	Reglas del negocio: validar stock, calcular totales	producto.servicio.js
modelos	Consultas SQL a MySQL con mysql2	producto.modelo.js
middlewares	Funciones que se ejecutan antes o después de las rutas	manejarErrores.js, verificarToken.js
errores	Clases de error propias	StockInsuficienteError.js
vistas	Plantillas EJS con formularios (AA2-EV02)	productos/lista.ejs

Nota: Los archivos que exportan una clase llevan el nombre de la clase en PascalCase: StockInsuficienteError.js.

5.2 Nombramiento de clases

- **PascalCase** y un **sustantivo** en singular: Producto, Pedido, DetallePedido.
- Las clases de error extienden de Error y terminan en **Error**: StockInsuficienteError, NoEncontradoError.
- Una clase por archivo, con el mismo nombre de la clase.
- Las clases se usan para entidades y errores. Rutas, controladores, servicios y modelos son módulos que exportan funciones.

5.3 Nombramiento de funciones y métodos

- **camelCase** y empiezan con un **verbo** en infinitivo: crear, buscar, listar, actualizar, eliminar, calcular, validar.
- Las funciones que devuelven boolean empiezan con **es, esta, tiene o puede**: estaAgotado(), tieneStock().
- Las funciones asíncronas se declaran con **async** y se llaman con **await**; no se mezclan con .then().
- Una función hace **una sola cosa** y, como guía, no pasa de 30 líneas.
- Si una función necesita más de tres parámetros, recibe un **objeto**: crearProducto({ nombre, precio, stock }).

Operación	Modelo (SQL)	Controlador	Ruta
Inserción	insertar(producto)	crearProducto	POST /api/productos
Consulta de uno	buscarPorId(idProducto)	obtenerProducto	GET /api/productos/:id
Consulta de todos	listarTodos()	listarProductos	GET /api/productos
Actualización	actualizar(idProducto, cambios)	actualizarProducto	PUT /api/productos/:id
Eliminación	eliminar(idProducto)	eliminarProducto	DELETE /api/productos/:id

5.4 Nombramiento de variables y constantes

- Variables en **camelCase** con un sustantivo descriptivo: precioUnitario, fechaPedido, usuarioActual.
- Los arreglos van en **plural**: productos, lineasPedido.
- Las variables de una letra solo se permiten como contador de un for corto (i, j).
- Las constantes del módulo van en **UPPER_SNAKE_CASE**: STOCK_BAJO = 5, MAX_INTENTOS_LOGIN = 3.
- Las variables de entorno también van en UPPER_SNAKE_CASE y se leen solo en config/: process.env.DB_HOST.

Incorrecto	Correcto	Por qué
var x = 5;	const cantidad = 5;	const por defecto y nombre que dice qué guarda
let NombreProd;	let nombreProducto;	camelCase y sin abreviar
if (stock < 5)	if (stock < STOCK_BAJO)	Sin números mágicos

Incorrecto	Correcto	Por qué
if (stock == "0")	if (stock === 0)	Comparación estricta
const lista = [];	const productos = [];	Plural que indica el contenido
function agotado()	function estaAgotado()	Los booleanos se leen como pregunta

5.5 Declaración de clases

Los miembros de una clase se declaran siempre en este orden, separados por una línea en blanco:

- **Primero:** constantes estáticas (static STOCK_BAJO).
- **Segundo:** campos privados con # cuando no deban modificarse desde fuera.
- **Tercero:** constructor.
- **Cuarto:** métodos estáticos de creación (desdeFila).
- **Quinto:** métodos públicos, agrupados por funcionalidad.
- **Último:** métodos privados de apoyo (#calcular…).

```
/**
 * Producto que se vende en la tienda.
 * Corresponde a la tabla productos de la base de datos deportes.
 */
export class Producto {
  static STOCK_BAJO = 5;

  constructor({ idProducto, nombre, precio, stock }) {
    this.idProducto = idProducto;
    this.nombre = nombre;
    this.precio = precio;
    this.stock = stock;
  }

  /** Crea un producto a partir de una fila de MySQL. */
  static desdeFila(fila) {
    return new Producto({
      idProducto: fila.Id_producto,
      nombre: fila.Nombre,
      precio: Number(fila.Precio),
      stock: fila.Stock,
    });
  }

  /** Indica si quedan pocas unidades para mostrar la alerta en el catálogo. */
  tieneStockBajo() {
    return this.stock > 0 && this.stock <= Producto.STOCK_BAJO;
  }

  estaAgotado() {
    return this.stock === 0;
  }
}
```

Las clases de error agregan el código HTTP que debe responder la API:

```
/** Se lanza cuando se pide más cantidad de la que hay en stock. */
export class StockInsuficienteError extends Error {
  constructor(nombreProducto) {
    super(`No hay stock suficiente de ${nombreProducto}.`);
    this.name = "StockInsuficienteError";
    this.estadoHttp = 409;
  }
}
```

5.6 Declaración de funciones y acceso a datos

- Las funciones exportadas se declaran con **function** (o async function) y llevan JSDoc; las funciones flecha se usan para callbacks cortos.
- Se usa un **pool** de mysql2 y consultas con **parámetros (?)**; nunca se concatenan valores en el SQL, así se evita la inyección SQL.

- Las consultas se guardan en constantes con prefijo SQL_ al inicio del módulo.
- Las credenciales se leen de variables de entorno (archivo .env que no se sube a GitHub).

```
// src/config/baseDatos.js
import mysql from "mysql2/promise";

export const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USUARIO,
  password: process.env.DB_CONTRASENA,
  database: process.env.DB_NOMBRE,
});
```

```
// src/modelos/producto.modelo.js
import { pool } from "../config/baseDatos.js";
import { Producto } from "./Producto.js";

const SQL_BUSCAR_POR_ID =
  "SELECT Id_producto, Nombre, Precio, Stock FROM productos WHERE Id_producto = ?";

/**
 * Busca un producto por su identificador.
 * @param {number} idProducto - Identificador del producto.
 * @returns {Promise<Producto|null>} El producto, o null si no existe.
 */
export async function buscarPorId(idProducto) {
  const [filas] = await pool.execute(SQL_BUSCAR_POR_ID, [idProducto]);
  if (filas.length === 0) {
    return null;
  }
  return Producto.desdeFila(filas[0]);
}
```

5.7 Rutas, controladores y manejo de errores en Express

- Cada recurso tiene su archivo de rutas, que exporta un Router con el nombre del recurso en plural: productosRutas.
- Los controladores se nombran **verbo + recurso** (obtenerProducto) y solo leen la petición, llaman al servicio y responden.
- Express 5 envía automáticamente al middleware de errores las promesas rechazadas, así que los controladores async no necesitan try/catch.
- Hay **un solo middleware de errores**, al final de app.js, que responde con el código HTTP y un mensaje claro.
- Nunca se deja un catch vacío ni se muestra al usuario el detalle técnico del error.

```
// src/rutas/producto.rutas.js
import { Router } from "express";
import * as productoControlador from "../controladores/producto.controlador.js";

export const productosRutas = Router();

productosRutas.get("/", productoControlador.listarProductos);
productosRutas.get("/:id", productoControlador.obtenerProducto);
productosRutas.post("/", productoControlador.crearProducto);
productosRutas.put("/:id", productoControlador.actualizarProducto);
productosRutas.delete("/:id", productoControlador.eliminarProducto);
```

```
// src/controladores/producto.controlador.js
import * as productoServicio from "../servicios/producto.servicio.js";

/** GET /api/productos/:id · Responde un producto o 404 si no existe. */
export async function obtenerProducto(req, res) {
  const idProducto = Number(req.params.id);
  const producto = await productoServicio.obtenerPorId(idProducto);
  res.json(producto);
}
```

```
// src/middlewares/manejarErrores.js
/** Último middleware de app.js: convierte cualquier error en una respuesta JSON. */
export function manejarErrores(error, req, res, next) {
  const estadoHttp = error.estadoHttp ?? 500;
  if (estadoHttp === 500) {
    console.error(error);
  }
  res.status(estadoHttp).json({
    mensaje: estadoHttp === 500 ? "Ocurrió un error en el servidor." : error.message,
  });
}
```

5.8 Comentarios

- **JSDoc** (/** … */) en toda clase y en toda función exportada: qué hace, @param y @returns.
- Los comentarios de línea (//) explican **por qué** se hace algo, no repiten lo que el código ya dice.
- La primera línea de cada archivo puede indicar su ruta, como en los ejemplos, para ubicarlo rápido.
- No se deja código comentado: para eso está el historial de Git.
- Las tareas pendientes se marcan con // TODO: y se registran como issue en GitHub.

6. Estándares para React

Elemento	Regla	Ejemplo
Componentes	PascalCase; un componente por archivo .jsx con el mismo nombre	TarjetaProducto.jsx
Hooks propios	Empiezan con use	useCarrito, useSesion
Manejadores de eventos	Prefijo manejar dentro del componente; prefijo on en las props	manejarClickAgregar, onAgregar
Booleanos	es, esta, tiene	estaCargando, tieneErrores
Llamadas a la API	Un archivo por recurso en services/	productoApi.js
Carpetas	minúsculas: components, pages, hooks, services, utils	src/components/TarjetaProducto.jsx
Componentes	Funciones, no clases; props desestructuradas	function TarjetaProducto({ producto })

```
import { formatearPesos } from "../utils/formato.js";

/** Tarjeta del catálogo con nombre, precio, stock y botón Agregar. */
export function TarjetaProducto({ producto, onAgregar }) {
  const estaAgotado = producto.stock === 0;

  function manejarClickAgregar() {
    onAgregar(producto.idProducto);
  }

  return (
    <article className="tarjeta-producto">
      <h3>{producto.nombre}</h3>
      <p>{formatearPesos(producto.precio)}</p>
      <button type="button" disabled={estaAgotado} onClick={manejarClickAgregar}>
        {estaAgotado ? "Agotado" : "Agregar"}
      </button>
    </article>
  );
}
```

7. Estándares para la base de datos

Las tablas y columnas nuevas se nombran en **snake_case**, en minúsculas, y la llave primaria se llama id_<tabla>. Las llaves foráneas usan el mismo nombre de la llave que referencian.

La base deportes creada en la Guía 6 mezcla estilos (Users, NOmnbre, Id_pedido). Para no romper lo ya construido se mantiene en la AA2 y se corrige en el primer sprint de la AA3 con un

script de migración versionado en database/. Mientras tanto, el código usa siempre nombres estándar y solo los modelos conocen los nombres actuales (lo hace Producto.desdeFila):

Nombre actual	Nombre estándar	Propiedad en JavaScript
Users	usuarios	Usuario (clase)
NOmnbre	nombre	nombre
Passwords	contrasena_hash	contrasenaHash
Id_pedido	id_pedido	idPedido
Id_producto	id_producto	idProducto

Nota: Passwords cambia a contrasena_hash porque las contraseñas no se guardan en texto plano: se guarda su hash con bcrypt.

8. Estándares para la API REST

- Las rutas usan **sustantivos en plural** y kebab-case, nunca verbos: /api/productos, /api/detalle-pedidos.
- La acción la indica el método HTTP: GET consulta, POST crea, PUT actualiza, DELETE elimina.
- Los atributos del JSON van en camelCase: { "idProducto": 1, "precio": 89900 }.
- Las respuestas usan el código HTTP correcto: 200, 201, 400, 401, 404, 409 y 500.

Método	Ruta	Acción
GET	/api/productos	Listar productos
GET	/api/productos/:id	Consultar un producto
POST	/api/productos	Crear un producto
PUT	/api/productos/:id	Actualizar un producto
DELETE	/api/productos/:id	Eliminar un producto
POST	/api/auth/login	Iniciar sesión

9. Herramientas para verificar los estándares

Herramienta	Qué verifica
EditorConfig	Indentación, codificación y fin de línea iguales en cualquier editor

Herramienta	Qué verifica
ESLint	Errores y reglas de estilo en Node.js y React (npm run lint)
Prettier	Da formato automático al guardar en VS Code
Jest y Supertest	Pruebas automáticas de servicios y endpoints (npm test)
GitHub Actions	Ejecuta npm run lint y npm test en cada push (AA1-EV01)

Archivo .editorconfig en la raíz del repositorio:

```
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
trim_trailing_whitespace = true
indent_style = space
indent_size = 2
max_line_length = 100
```

Scripts en backend/package.json para ejecutar las verificaciones:

```
{
  "type": "module",
  "scripts": {
    "dev": "node --watch --env-file=.env servidor.js",
    "start": "node --env-file=.env servidor.js",
    "lint": "eslint .",
    "format": "prettier --write .",
    "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js"
  }
}
```

Nota: Jest necesita la opción --experimental-vm-modules para trabajar con módulos ES.

10. Lista de verificación antes de cada commit

- Las carpetas, archivos, clases, funciones y variables siguen las reglas de nombramiento.
- No hay var, ni ==, ni números mágicos, ni variables de una letra fuera de los bucles.
- Las clases y funciones exportadas tienen JSDoc.
- El SQL usa parámetros (?) y las credenciales vienen del archivo .env.
- No hay contraseñas ni datos personales en el código.
- npm run lint y npm test terminan sin errores.

- El mensaje del commit sigue Conventional Commits.

11. Conclusiones

Los estándares definidos dan una forma única de escribir el código de la Tienda Deportes. Como el back-end y el front-end usan JavaScript, las mismas reglas de nombres y formato sirven en Node.js, Express y React. Nombrar en español y con reglas fijas hace que el código se lea igual que los requerimientos, y la organización por capas permite saber qué hay en cada archivo con solo ver su nombre.

EditorConfig, ESLint, Prettier y las pruebas con Jest convierten las reglas en verificaciones automáticas, de modo que el estándar se cumple sin depender de la memoria. Estas reglas se aplicarán desde el primer módulo de la AA2-EV01, que la guía evalúa justamente por el nombramiento de variables, métodos, clases y paquetes.

12. Referencias

Airbnb. (2024). Airbnb JavaScript Style Guide. https://github.com/airbnb/javascript

Goldberg, Y. (2024). Node.js Best Practices. https://github.com/goldbergyoni/nodebestpractices

OpenJS Foundation. (2026). Express: guía de enrutamiento y manejo de errores. https://expressjs.com/es/

React. (2026). Documentación de React. https://es.react.dev/

Holywell, S. (2023). SQL Style Guide. https://www.sqlstyle.guide/es/

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

SENA. (2024). Guía de aprendizaje 7: Codificar los módulos del software. Programa ADSO 228118.